

Implementation and Comparison of RNN, LSTM, and GRU

Sequence Classification Assignment

1. Objective

To implement Simple RNN, LSTM, and GRU deep learning models for sequence classification and compare their performance using appropriate evaluation metrics.

2. Dataset Used

The IMDB Movie Review Sentiment Dataset is used. It contains movie reviews labeled as positive or negative, making it suitable for binary sequence classification.

Dataset	IMDB Movie Review Sentiment Dataset
Task	Binary Sentiment Classification
Classes	0 = Negative, 1 = Positive
Vocabulary Size	10,000 most frequent words
Maximum Sequence Length	200 tokens
Training Samples	25,000
Testing Samples	25,000

3. Theory

Recurrent Neural Networks (RNNs) are neural networks designed to process sequential data. At each time step, the network uses the current input and information from the previous hidden state. A basic RNN can suffer from vanishing gradients when learning long-term dependencies.

Long Short-Term Memory (LSTM) networks extend RNNs with a memory cell and gates. The forget, input, and output gates control which information is discarded, stored, and passed forward.

Gated Recurrent Units (GRUs) are another gated recurrent architecture. They use update and reset mechanisms and generally have a simpler structure than LSTM.

4. Methodology

1. Load the IMDB dataset using TensorFlow/Keras.
2. Restrict the vocabulary to the 10,000 most frequent words.
3. Pad or truncate each review to a fixed length of 200 tokens.
4. Build a Simple RNN model.
5. Build an LSTM model.
6. Build a GRU model.

7. Train all models using Adam optimizer and binary cross-entropy loss.
8. Use early stopping based on validation loss.
9. Generate predictions on the test dataset.
10. Calculate accuracy, precision, recall, and F1 score.
11. Plot validation accuracy and loss for comparison.
12. Generate confusion matrices and compare trainable parameter counts.

5. Model Architectures

Model	Main Layer	Additional Layers	Output
RNN	SimpleRNN(64)	Embedding → Dropout → Dense(32)	Sigmoid
LSTM	LSTM(64)	Embedding → Dropout → Dense(32)	Sigmoid
GRU	GRU(64)	Embedding → Dropout → Dense(32)	Sigmoid

6. Important Code

```
# Example LSTM model
model = Sequential([
    Embedding(10000, 64, input_length=200),
    LSTM(64),
    Dropout(0.3),
    Dense(32, activation="relu"),
    Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model.fit(
    X_train,
    y_train,
    epochs=8,
    batch_size=128,
    validation_split=0.2
)
```

7. Evaluation Metrics

Accuracy is the proportion of correctly classified samples out of all samples.

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Precision measures how many samples predicted as positive are actually positive.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall measures how many actual positive samples are correctly identified.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

F1 score is the harmonic mean of precision and recall and is useful when both types of classification error matter.

$$\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

8. Performance Analysis

The notebook calculates all four metrics on the same test set for RNN, LSTM, and GRU. It also plots validation accuracy and validation loss, displays confusion matrices, and compares the number of trainable parameters.

The exact numerical results are generated when the notebook is executed because neural-network training can vary slightly between runs.

9. Expected Observations

- Simple RNN has the simplest recurrent architecture and is computationally straightforward.
- LSTM is designed to preserve useful information over longer sequences using memory cells and gates.
- GRU provides gated memory with a simpler architecture than LSTM.
- Accuracy alone should not be used to judge a classifier; precision, recall, F1 score, and the confusion matrix provide additional information.
- The model with the best metric values on the test set should be identified from the actual results produced by the notebook rather than assumed in advance.

10. Advantages and Limitations

RNN:

- Simple architecture.
- Suitable for basic sequential tasks.
- Can have difficulty with long-term dependencies.

LSTM:

- Handles long-term dependencies effectively.
- Uses explicit memory cells and gates.
- Has more parameters and computational cost than a basic RNN.

GRU:

- Uses fewer gating components than LSTM.

- Often provides a useful balance between model complexity and sequence modeling ability.
- May be effective when faster or simpler recurrent modeling is desired.

11. Conclusion

Simple RNN, LSTM, and GRU models were successfully implemented for binary sentiment classification using the IMDB dataset. The models were trained under comparable conditions and evaluated using accuracy, precision, recall, F1 score, validation curves, and confusion matrices. The experiment demonstrates the differences between basic recurrent networks and gated recurrent architectures and provides a practical method for comparing sequence-classification models.